

IFC2LBD-Neo: A Rust-Based Converter for Efficient Linked Building Data Generation from IFC

Lukas Kirner, Jyrki Oraskari, Alex Donkers, Ekaterina Petrova, Pieter Pauwels, Jakob Beetz
Eindhoven University of Technology, RWTH Aachen University
l.t.kirner@tue.nl

Abstract. A key barrier to interoperability in Building Information Modeling (BIM) is the efficient transformation of Industry Foundation Classes (IFC) into web-compatible, linkable, and queryable representations. This paper presents IFC2LBD-Neo¹, a Rust-based converter for generating Linked Building Data (LBD) and linked ifcOWL from IFC models. Building on the IFCToLBD lineage, the system introduces a staged pipeline architecture enabling streaming, parallel processing, and scalable RDF serialization. The converter supports IFC2X3 to IFC4X3 and produces Turtle or chunked N-Quads output for incremental ingestion. Results show substantial performance improvements over the Java-based predecessor, reducing runtime by up to 20× in controlled benchmarks, with larger gains in large-model scenarios, reducing peak memory consumption by more than 8× while preserving output compatibility. In addition, a multi-stage topology extraction approach combines lightweight inference with selective geometry-based refinement to improve topology completeness. IFC2LBD-Neo provides an efficient and extensible solution for large-scale IFC-to-LBD workflows in data-intensive BIM and digital twin applications.

1 Introduction

Interoperability remains one of the central challenges in construction, particularly in the context of Building Information Modeling (BIM), where large volumes of structured building data are created and shared among project stakeholders. Building information must move across authoring tools, coordination environments, analysis workflows, and operational systems. This requires data that can be exchanged across heterogeneous software landscapes while remaining usable for further processing (Eastman et al., 2018). The Industry Foundation Classes (IFC) standard provides the main neutral exchange model for this purpose (Borrmann et al., 2018). However, neutral exchange formats alone do not ensure effective web-based representation, linking, or querying of building information. As data must be integrated across heterogeneous systems without a shared software stack, semantics, interoperability, and machine interpretability are essential to preserve meaning and enable automated integration. This is particularly relevant for digital twins, where building data must be integrated with external information that cannot be fully represented within IFC (Kirner et al., 2024). Semantic Web and Linked Data approaches in architecture and construction address this gap. The ifcOWL effort maps IFC into the Web Ontology Language (OWL) and the Resource Description Framework (RDF), enabling semantically explicit representations and facilitating querying, reasoning, and linking of IFC-based information (Beetz et al., 2009; Pauwels and Terkaj, 2016). Software support for this transformation was provided by IFCToRDF, which offered reusable Java components for IFC-to-RDF conversion and formed an important technical basis (Pauwels, 2020). Building on this, IFCToLBD showed that IFC models can be converted into more lightweight and modular Linked Building Data (LBD) graphs that are easier to query in many practical scenarios (Bonduel et al., 2018). This contribution was significant both for its resulting graph structure and for lowering the barrier to adoption of LBD and ifcOWL. The Java-based IFCToLBD software line, therefore, remains the direct implementation baseline for the present work (Oraskari et al., 2026). The

¹<https://github.com/ISBE-TUE/IFC2LBD-Neo>

motivation for a novel converter iteration is primarily practical. Large IFC models exceeding 100 MB introduce substantial runtime and hardware challenges, particularly since conversions can produce very large RDF graphs depending on the desired depth of information. Additionally, topology outputs expressed with the Building Topology Ontology (BOT) (Rasmussen et al., 2021) are often incomplete, as IFC input does not always contain all relations needed to derive richer topology graphs directly. Current conversion workflows lack efficient solutions that address both performance constraints and completeness of topology. The objective is to carry the original converter concept forward by providing an implementation better aligned with current performance, topology, and output requirements, while retaining full ifcOWL compatibility. Therefore, this paper presents IFC2LBD-Neo, a Rust-based converter for efficient Linked Building Data generation from IFC. It is developed in the IFctoLBD lineage, but with a new implementation architecture designed to improve speed and reduce hardware requirements. The paper treats the Java converter as the reference implementation, where meaningful, while also examining the engineering advantages and current limitations of the new system.

2 Design Goals

The design of IFC2LBD-Neo is based on findings gathered while using the IFctoLBD converter in construction projects. In such settings, models are often large, heterogeneous in quality, and inconsistent in their adherence to IFC semantics. The four goals below translate these observations into requirements:

Scalability and Resource Use. Large IFC models, especially those exceeding 100 MB, can cause substantial memory pressure and long runtimes during conversion. In earlier IFctoLBD workflows, this was aggravated by execution paths that materialized large triple sets in memory, while the core conversion flow remained largely sequential. In observed workflows, runtime exceeded 60 minutes depending on hardware, and occasional Java out-of-memory failures occurred on project-scale inputs. The legacy tool has remained under active development through 2026, including TDB-backed improvements for larger models (Oraskari et al., 2026). IFC2LBD-Neo nevertheless follows the same central requirement. Conversion should reduce peak resource use, improve turnaround time, and handle large outputs more efficiently.

Topology Coverage. Topology output is constrained by the quality and completeness of the IFC input itself, and many models do not contain all explicit relations needed to derive a richer BOT graph directly. IFctoLBD exports BOT statements such as `bot:containsElement` and `bot:hasSubElement`, with optional bounding-box-based enrichment including `bot:Interface` resources. For a more complete BOT graph, IFC2LBD-Neo uses a staged topology pipeline that relies on lightweight IFC evidence first and escalates only where needed to more expensive geometric checks, including a full OpenCascade (OCC)-based path. This keeps the derivation logic explicit and avoids imposing full geometric processing on every conversion run.

Dual Output Compatibility. A major strength of IFctoLBD is that it does not force a choice between concise LBD output and linked ifcOWL output, which support different downstream tasks. LBD improves direct usability, while ifcOWL remains important for compatibility, traceability, and schema-oriented access. IFC2LBD-Neo, therefore, treats this dual output as a requirement rather than an option, preserving linked ifcOWL output without simplifying the workflow at the cost of compatibility.

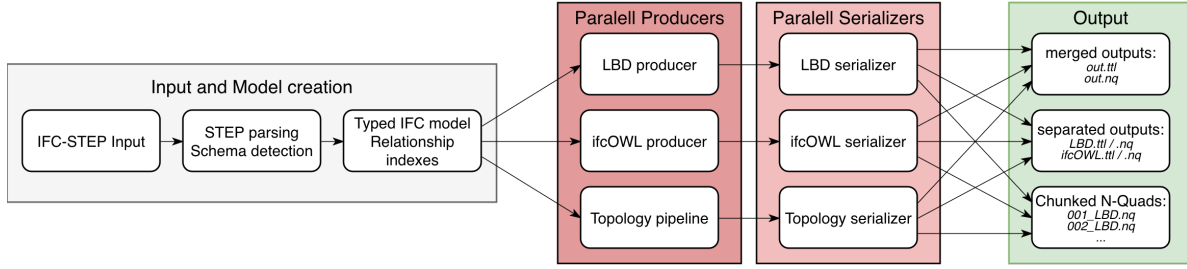


Figure 1: System architecture and processing modes of IFC2LBD-Neo, including input, parallel producers, serializers and output.

Controlled Extensibility. The legacy converter accumulated a wide range of flags and output options because different projects require different semantic depths, serializations, and enrichment steps, indicating that the converter will continue to evolve. IFC2LBD-Neo, therefore, needs a structure that allows new built-in flags, topology modes, and conversion stages to be added without obscuring the pipeline. The goal is controlled, modular extensibility while keeping the main conversion stages clearly separated and the output predictable as new capabilities are added.

3 System Architecture and Processing Modes

IFC2LBD-Neo is implemented in Rust and structured as a staged conversion pipeline. Rust is chosen for its ability to combine fine-grained control over memory-intensive operations with strong compile-time guarantees for safe concurrency (Klabnik and Nichols, 2018). The modular pipeline architecture supports a clear separation of parsing, modeling, conversion, topology, and serialization concerns. Figure 1 summarizes the simplified main processing path with its different stages and parallel execution pipelines.

3.1 STEP Parsing and Typed Model Construction

The input layer is split between STEP parsing and typed model construction. The parser reads the IFC STEP file and normalizes entity data, including schema detection from the file header. The current implementation supports IFC2X3, IFC4, IFC4X1, and IFC4X3, with IFC4X2 normalized to the IFC4X3 path. The model layer constructs a typed in-memory projection of the conversion-relevant IFC content. This `IfcModel` contains spatial and element nodes, property and quantity structures, and relationship indexes for containment, aggregation, hosting, fillings, voids, and space boundaries. It is a conversion-oriented representation of the subset required by downstream stages, rather than a full, generic IFC object model. This is more memory-efficient than building a full ifcOWL graph as an intermediate structure and repeated traversal of raw STEP records. The same internal model is reused for LBD conversion, topology derivation, and geometry-backed enrichment.

3.2 Parallel Producers

Once the typed model has been constructed, the converter enters the producer stage. In the streaming path, LBD triples are emitted in bounded batches through crossbeam channels, while ifcOWL triples can be emitted concurrently in separate threads. In the chunked N-Quads mode shown in Figure 1, topology can also be produced as a separate stream in parallel with the main conversion flow. The producer stage consists of several triple-producing paths that reuse the same internal model under different output and topology settings.

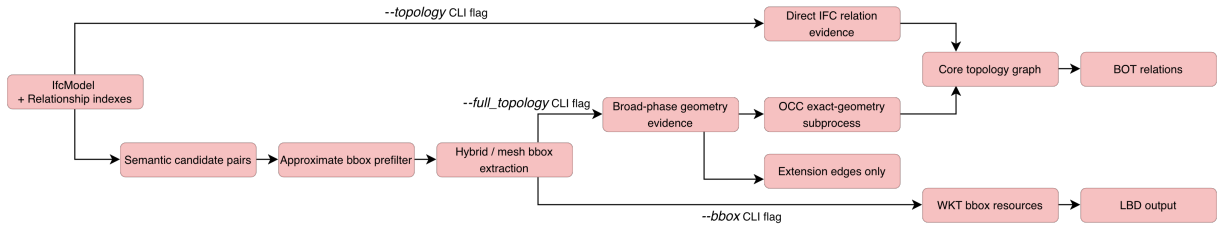


Figure 2: Left-to-right view of the staged topology escalation path.

3.3 Serializers and Chunk Writers

The serializer stage consumes the producer streams and turns them into RDF output files. In Turtle mode, the primary output is the LBD graph, and linked ifcOWL can be written as a separate file. ifcOWL Turtle is written incrementally to keep peak memory usage low for large outputs, while LBD Turtle collects triples for grouped, subject-oriented writing, trading readability for reduced streaming overhead. In N-Quads mode, the converter emits named graphs for LBD and ifcOWL, and optionally topology. Chunking by line count, byte threshold, or available CPU cores is supported, with each stream written through its own chunk-writer path and producing a manifest with file order and chunk statistics. In ingestion pipelines using triplestores that support N-Quads, chunking supports incremental loading, which promises more efficient handling of very large graph exports without semantic changes in graph content.

3.4 Topology Modes

Topology processing is optional and has two modes. The lightweight `-topology` mode derives topology from IFC relation evidence already present in the model, such as spatial containment, adjacency signals from space boundaries, and hosted or aggregated element relationships. The `-topology-full` mode extends this with staged geometric escalation. The first step in that path is candidate generation. If `IfcRelSpaceBoundary` evidence is available, candidate pairs are derived from spaces and their adjacent elements. If such evidence is missing, the Command-Line Interface (CLI) logic falls back to structural candidates and filters them using approximate STEP-derived bounding boxes. This keeps the exact-geometry phase focused on a reduced set of plausible pairs instead of the full cross-product of elements. Bounding boxes appear in two different roles. When `-bbox` is enabled, they are emitted as geometry resources with GeoSPARQL (Nicholas J. Car et al., 2024) Well-Known-Text (WKT) in the LBD output. In the topology processing described here, they serve as broad-phase filters that reduce the number of pairs passed to the expensive geometry stage. In the final stage, the full topology mode computes improved mesh-based world bounding boxes and passes reduced candidate sets to the OCC kernel, where exact boolean intersection operations are performed. Bbox-only relations are not emitted as final BOT statements but retained as internal evidence, while emitted intersections and interfaces are OCC-confirmed. Figure 2 summarizes this escalation path.

3.5 Modular Structure and Extension

IFC2LBD-Neo is organized as a Rust workspace with separate crates for STEP parsing, typed IFC modeling, topology derivation, geometry-backed enrichment, conversion, serialization, and CLI orchestration. This split is the basis for extending the converter without merging new concerns back into a single export routine. The converter layer already follows an emitter-based pattern in which LBD concerns can be extracted into dedicated modules while sharing the same buffered and streaming paths. This makes it feasible to add new built-in flags, output modes, and enrichment stages.

Table 1: Overview of the IFC models used in the evaluation and their input file sizes.

Model	IFC schema	Size (MB)	Role
DigitalHub_FM-ARC_v2.ifc	IFC4	9.0	baseline and mode comparison
large-model(non-disclosed)	IFC4x2	171.1	scalability
Wohn-Geschaefthaus.ifc	IFC2x3	8.5	bbox escalation case

4 Evaluation and Benchmarks

4.1 Evaluation Goals

The evaluation focuses on three aspects of IFC2LBD-Neo. First, it assesses runtime, memory use for conversion in comparison to the Java-based IFCtoLBD converter, as well as strict ifcOWL export completeness. Second, it examines the efficient handling of large IFC models, with particular attention to large RDF generation and chunked N-Quads output. Third, it evaluates the computational cost and practical behavior of the available topology generation modes, including lightweight topology extraction and the full topology path with bbox-supported escalation.

4.2 Evaluation Setup and Benchmark Models

The benchmarks are executed on a Debian 12 server equipped with an AMD 9955HX CPU, 64 GB of RAM, and 16 visible physical CPU cores. Runtime and peak Resident Set Size (RSS), the amount of physical memory a process occupies at a given time, are measured with `/usr/bin/time -v`. For the large-model study, IFC2LBD-Neo is run with 1, 4, and 16 cores (double threads) to examine scalability and how thread count affects high-output conversion. The baseline comparison between Rust and Java uses comparable Turtle plus linked ifcOWL output, while additional Rust-only runs additionally examine chunked N-Quads output as well as lightweight and full topology processing. Three IFC models are used, each with a distinct role. The primary comparison model is the openly available IFC4 dataset `DigitalHub_FM-ARC_v2.ifc` from the DigitalHub repository². It is a compact IFC-STEP model and is used for the direct Rust-versus-Java comparison, the ifcOWL completeness check, and the DigitalHub mode comparison. Its LBD output is not treated as a strict one-to-one comparison because several details of the Rust converter intentionally differ from the Java baseline. As a large-model case, a non-disclosed IFC4 coordination model with an input size of about 171 MB is used to study large-model efficiency, chunked N-Quads overhead, and the effect of different thread counts under a high-output workload. The third model is `Wohn-Geschaefthaus.ifc`³. It is used as a focused geometry case because diagonal and rounded elements make it suitable for checking the bbox escalation logic and the exact path for problematic boxes.

4.3 Performance Comparison

For the direct baseline comparison on `DigitalHub_FM-ARC_v2.ifc`, both converters were executed with LBD and linked ifcOWL output in Turtle. Table 2 summarizes the resulting runtimes, memory use, and total RDF output sizes. Across 20 repeated runs, the standard deviations are small relative to the means, indicating stable benchmark behavior on the selected server hardware. In the baseline comparison, IFC2LBD-Neo completes the DigitalHub conversion substantially faster than the Java converter, with much lower peak memory use. The mean wall time is 31.82 s for Java and 1.59 s for Rust, while the mean peak RSS, the resident physical memory of the process, is 3708.0 MB for Java and 439.7 MB for Rust. The corresponding error

²<https://github.com/RWTH-E3D/DigitalHub>

³<https://bingk.de/wp-content/uploads/2022/09/Wohn-Geschaefthaus.zip>

Table 2: DigitalHub benchmark results reported as mean $\pm$ standard deviation over $n = 20$ runs. Output size is the combined size of the generated RDF files for the respective run.

Configuration	Wall (s)	Peak RSS (MB)	Output (MB)
Java LBD + ifcOWL (TTL)	31.82 ± 0.25	3708.0 ± 247.2	147.50 ± 0.00
Rust LBD + ifcOWL (TTL)	1.59 ± 0.02	439.7 ± 4.5	178.70 ± 0.01
Rust LBD + ifcOWL (N-Quads chunked)	1.64 ± 0.07	306.0 ± 8.4	432.93 ± 0.01
Rust LBD + ifcOWL + topology	1.65 ± 0.03	444.2 ± 8.2	178.71 ± 0.01
Rust LBD + ifcOWL + full topology	7.48 ± 0.03	459.9 ± 8.0	180.20 ± 0.02

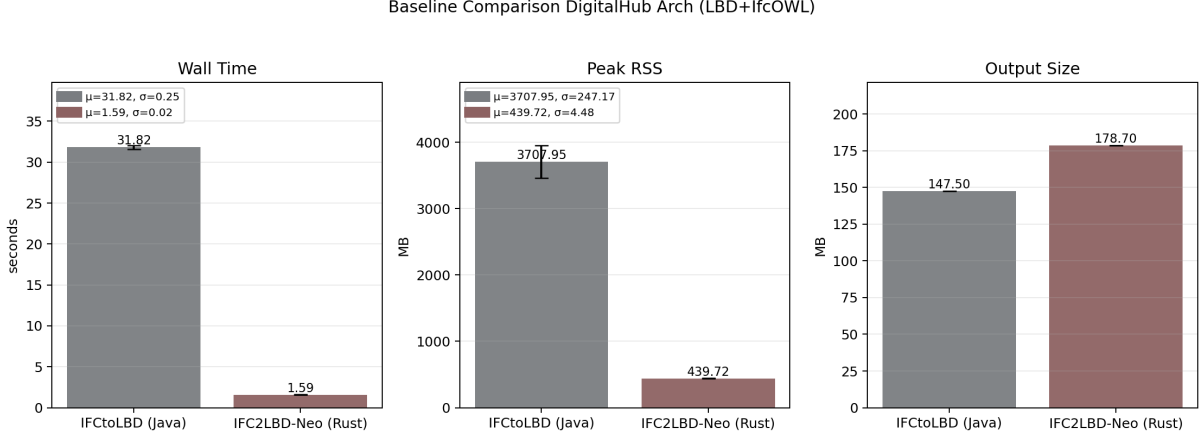


Figure 3: DigitalHub baseline comparison between the Rust and Java converters, shown as mean values with standard deviation error bars over 20 repeated runs.

bars in Figure 3 remain small, so the observed difference is clearly larger than run-to-run noise. For output size, the standard deviation is negligible and can be treated as effectively deterministic for these runs. Only the direct outputs of the ifcOWL conversions are lexically identical. The normalized LBD comparison on this IFC4 model reports `result=different` with dominant differences in property-set grouping. A strict one-to-one reproduction of the Java output would, in principle, be possible but would require aligning these design choices, thereby limiting some of the architectural flexibility explored here. The almost similar size of the baseline and lightweight topology outputs is explained by the limited additional relations in this model, as missing `IfcRelSpaceBoundary` restricts topology enrichment to a small set of basic relations, underscoring the need for the full topology pipeline.

4.4 Mode Comparison

The repeated mode comparison shows three stable effects. First, plain Turtle plus linked ifcOWL, chunked N-Quads, and lightweight topology remain close in runtime, all lying between 1.59 s and 1.65 s. Second, chunked N-Quads reduces peak RSS noticeably compared with plain Turtle while remaining nearly as fast, dropping from 439.7 MB to 306.0 MB on average. Third, `-topology` adds only a small overhead relative to the plain Turtle baseline, whereas the full topology mode introduces the only major runtime premium in this series. The full topology path rises to 7.48 s, while the lighter modes remain near the 1.6 s range. Here again, the standard deviations are small relative to the means. This indicates that the differences between the lightweight modes and the full topology mode are stable rather than caused by measurement noise. Architecturally, this is expected as relation-based topology is inexpensive, chunked N-Quads reduces memory usage, and full topology incurs additional geometric cost.

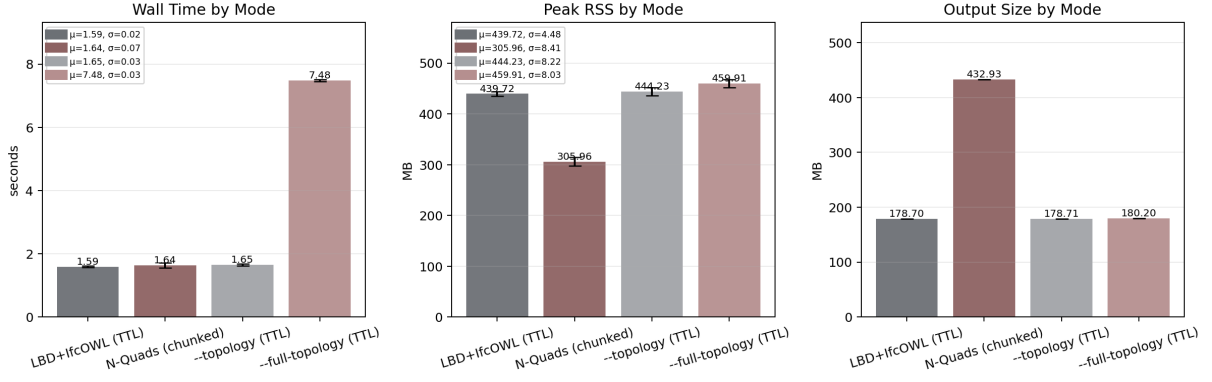


Figure 4: DigitalHub Rust mode comparison, shown as mean values with standard deviation error bars over 20 repeated runs.

4.5 Scalability Evaluation

The large-model study uses the 171.1 MB coordination model to examine runtime, peak RSS, and output growth across TTL and chunked N-Quads configurations under increasing core counts. In contrast to the repeated DigitalHub benchmark, this part of the evaluation is currently a controlled single-run per configuration. The main result is that the large-model behavior is not dominated by strong linear thread scaling. Moving from 1 core to 2 or 4 cores improves runtime clearly, but the gains flatten afterwards for most series. This suggests that the large-model path on the tested server is constrained at least in part by output and I/O costs rather than by compute alone. For the following comparisons, a 4-core / 8-thread setup is used because it is also a commonly available end-user configuration.

The format effect is also mode-dependent. For plain LBD + ifcOWL, TTL remains cheaper than chunked N-Quads at 4 cores / 8 threads. Plain TTL records 37.58 s and 3815 MB, whereas chunked N-Quads records 54.66 s and 4563 MB. For context, a single IFCtoLBD (Java) reference run of LBD + ifcOWL (TTL) on the same model and hardware required 1 h 37 min 13 s and reached about 24.9 GB peak RSS. The same pattern appears for -topology, where TTL records 38.12 s and 3772 MB, compared with 55.58 s and 4100 MB for chunked N-Quads. The full-topology case behaves differently. At 4 cores / 8 threads, chunked N-Quads completes in 99.05 s, while TTL requires 117.64 s. Once exact-kernel topology work dominates, the serialization penalty of N-Quads no longer determines the overall runtime.

Although chunked N-Quads increases output size and often also wall time during export, it remains relevant because it changes how large RDF output can be consumed downstream. In the representative DigitalHub run, the core-based policy produces 3 ifcOWL chunks and 1 LBD chunk. On the large model at 4 cores / 8 threads, plain chunked N-Quads produces 4 LBD chunks and 9 ifcOWL chunks. These files can be consumed incrementally by downstream ingestion pipelines while conversion is still in progress. This is particularly relevant for triplestores such as Oxigraph⁴ that support bulk loading of N-Quads. The present paper does not benchmark ingestion directly, but this setup strongly suggests an operational advantage for chunked N-Quads in end-to-end workflows, even when the export step alone is slower than TTL. The benefit is therefore operational rather than purely computational: chunked N-Quads expose the graph as a stream of manageable files, reducing the need to wait for a single monolithic RDF export before ingestion can begin.

⁴<https://github.com/oxigraph/oxigraph>

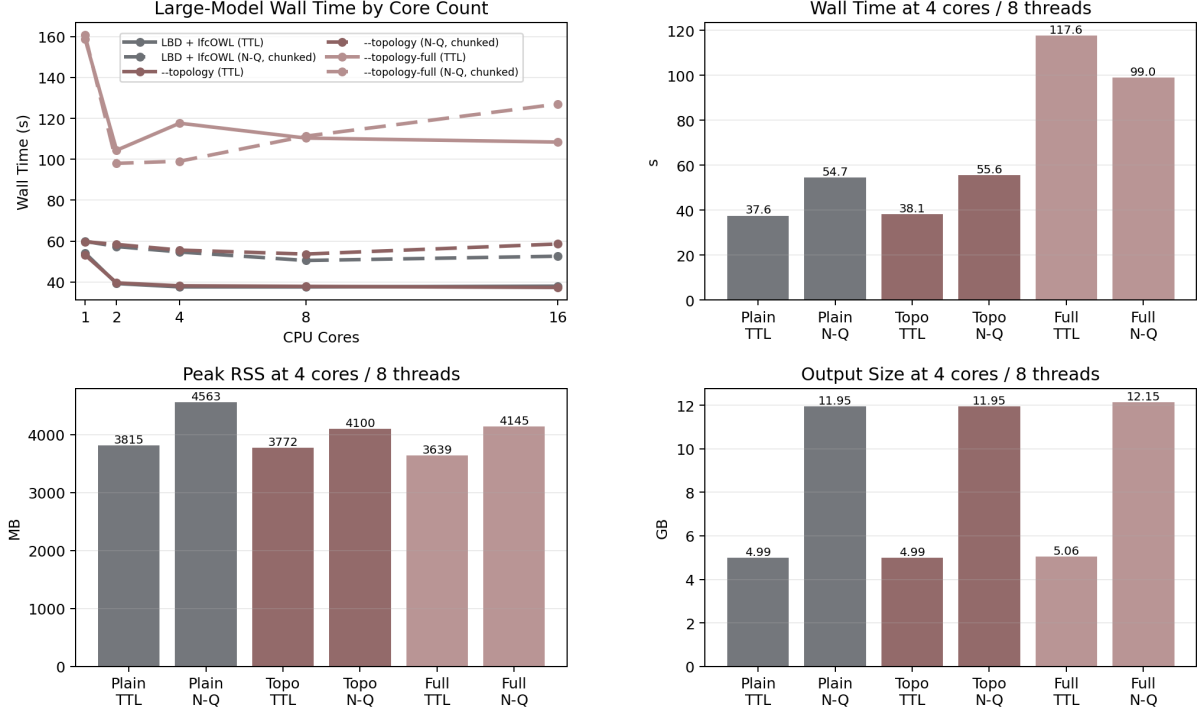


Figure 5: Large-model runtime, peak RSS, and output-size comparison across TTL and chunked N-Quads configurations. The upper-left panel shows wall-time trends over increasing core counts, while the other panels summarize the 4-core / 8-thread comparison used in the text.

4.6 Topology and Bounding-Box Observations

The model `Wohn-Geschaftshaus.ifc` is used to evaluate the selectivity and effect of the full-topology pipeline on geometry that is difficult for lightweight relation-based topology alone. In this model, no candidates can be derived from `IfcRelSpaceBoundary`. All 43,435 topology candidates come from the fallback logic. The bbox broad phase filters out 41,935 of these pairs, so only 1,500 pairs are sent to the exact geometric stage. This is the key selectivity result: The expensive stage is applied only to a small fraction of the initial candidate set.

The exact stage confirms 1,301 relations. Lightweight `-topology` produces 1,409 topology triples, whereas `-topology-full` produces 5,299. The full-topology output therefore adds 3,890 topology triples, including 1,288 `rdf:type bot:Interface` triples, 2,576 `bot:interfaceOf` triples, and 26 `bot:intersectingElement` triples. In lightweight mode, these geometry-derived interface and intersection statements are absent.

Bounding-box inflation remains a useful diagnostic for why escalation is needed. Here, inflation denotes the ratio between the fast axis-aligned bounding box and the tighter rotated bounding box used in the exact path. A value of 1.0, therefore, means that the fast box is already tight, while a threshold of 1.5 means that the axis-aligned box overestimates the element by at least 50% and should be refined. Of the 1,126 analyzed elements, 16 exceeded this threshold, and all 16 were routed through the exact rotated-bbox path. The strongest outlier had inflation of 17.10, which dropped to 1.00 after escalation, while the worst final inflation across the run remained 1.41. The resulting topology triples appear plausible for these cases, but their full validity and robustness should be examined further in future development.

5 Discussion, Limitations, and Future Work

The evaluation shows that IFC2LBD-Neo is not simply a direct language port of the Java-based IFCtoLBD converter, but represents a practically different implementation architecture centered on streaming-oriented processing, staged transformation, and explicit separation of conversion concerns. In the benchmark based on the openly available IFC4 DigitalHub model, the Rust implementation significantly reduces runtime and peak memory while preserving the dual-output model of LBD and linked ifcOWL. In the large-model study, the converter efficiently handles very large RDF outputs in both Turtle and chunked N-Quads modes, achieving low gigabyte RSS and a fraction of the original conversion times. The full-topology path produces substantially more interface and intersection statements than lightweight relation-based extraction alone, demonstrating that staged enrichment combining semantic evidence with selective geometric refinement can increase topology completeness without incurring the full cost of exhaustive geometry processing. In conclusion, these results support the main contribution of the paper, showing that the converter direction established by IFCtoLBD remains scientifically and semantically valid, while current usage scenarios benefit from a more efficient and scalable implementation approach.

At the same time, the results also show the limits of what this evaluation can demonstrate. First, the software does not yet reach full semantic parity for all outputs. The ifcOWL comparison remains the strongest baseline for completeness, whereas LBD output differs intentionally from the Java implementation. Second, the large-model study fails to present evidence of strong linear multicore scaling. The measured gains flatten after a few cores suggesting constraints by output and I/O behavior. Third, the large-model matrix is currently a controlled single-run per configuration, not a repeated-run variance study. Finally, the topology results are encouraging, but the validity and robustness of the generated topology triples still require broader validation across additional models and edge cases. Generally, the evaluation highlights that different processing modes reflect trade-offs rather than universally optimal choices, for example, between memory consumption, runtime, and downstream ingestion behavior.

These limitations also define the most relevant next steps. A key priority is broader evaluation, including additional public IFC models, more repeated runs for the large-model series, and direct end-to-end ingestion measurements with triplestores such as Oxigraph rather than singular exports. Strengthening topology validation is equally important, particularly for difficult geometric cases such as rounded shapes and for relation types introduced by the full-topology path. In addition, further architectural and semantic modularization is needed to support extension of the underlying data model and integration with additional ontologies and geometry representations. The current Rust workspace already supports the addition of new built-in conversion stages, output options, and enrichment steps, but a more explicit extension mechanism remains future work. This includes clearer module boundaries for new flags and, in the longer term, support for third-party conversion modules. Deployment-oriented extensions are also important future work. Chunked N-Quads already points toward pipeline-style ingestion workflows, and this should be complemented by more direct support for service and database integration. In addition, WebAssembly-based deployment remains an attractive direction for browser-side embedded use cases where local IFC inspection and selective conversion could benefit from the same typed-model and staged-processing architecture. In the longer term, this work also points toward incremental and streaming-based IFC processing, where building models are continuously transformed into queryable knowledge graphs for integration with digital twins and real-time data environments. These points are not required to validate the present paper, but they show that IFC2LBD-Neo is positioned not only as a converter implementation, but also as a basis for further research and engineering development.

6 Conclusion

This paper presented IFC2LBD-Neo as a Rust-based continuation of the IFctoLBD converter line for Linked Building Data generation from IFC. Within the evaluated benchmark scope, the results show that the new implementation achieves up to 20× faster runtime and reduces peak memory consumption by more than 8×, while preserving linked ifcOWL output alongside LBD generation and supporting a staged topology pipeline that materially extends topology output on difficult geometric cases. A large-model study further shows that very large RDF exports can be realistically handled in a controlled way, even when the main advantage of chunked N-Quads lies in operational output handling rather than uniformly lower export time. IFC2LBD-Neo should therefore be understood as a direct continuation and a compatible successor of the original converter contribution and as a more efficient architectural basis for future conversion workflows from IFC-Step to Linked Building Data.

7 Acknowledgements

This work has received funding from the European Union’s Horizon Europe research and innovation programme under the Pathfinder project "Additive To Predictive Manufacturing For Multistorey Construction Using Learning By Printing And Networked Robotics - AM2PM" (Grant Agreement No. 101162318). This work was partly funded by the German Research Foundation (DFG) under the Cluster of Excellence CARE: Climate-Neutral And Resource-Efficient Construction (EXC 3115), project number 533767731.

References

- Beetz, J., van Leeuwen, J., and de Vries, B. (2009). Ifcowl: A case of transforming EXPRESS schemas into ontologies. *Artificial Intelligence for Engineering Design, Analysis and Manufacturing*, 23(1):89–101.
- Bonduel, M., Oraskari, J., Pauwels, P., Vergauwen, M., and Klein, R. (2018). The IFC to linked building data converter: Current status. In *Proceedings of the 6th Linked Data in Architecture and Construction Workshop*, volume 2159 of *CEUR Workshop Proceedings*, pages 34–43, London, United Kingdom.
- Borrmann, A., Beetz, J., Koch, C., Liebich, T., and Muhic, S. (2018). Industry foundation classes: A standardized data model for the vendor-neutral exchange of digital building models. In *Building Information Modeling: Technology Foundations and Industry Practice*, pages 81–126. Springer, Cham.
- Eastman, C., Teicholz, P., Sacks, R., and Lee, G. (2018). *BIM Handbook: A Guide to Building Information Modeling for Owners, Designers, Engineers, Contractors, and Facility Managers*. Wiley, Hoboken, New Jersey, 3 edition.
- Kirner, L., Jung, V., Oraskari, J., and Brell-Cokcan, S. (2024). Enhancing robotic steel prefabrication with semantic digital twins driven by established industry standards. *Automation in Construction*, 167:105699.
- Klabnik, S. and Nichols, C. (2018). *The Rust Programming Language*. No Starch Press.
- Nicholas J. Car, Timo Homburg, Matthew Perry, John Herring, Frans Knibbe, Simon J.D. Cox, Joseph Abhayaratna, and Mathias Bonduel (2024). OGC GeoSPARQL - A Geographic Query Language for RDF Data. OGC Implementation Standard OGC 22-047, Open Geospatial Consortium.
- Oraskari, J., Bonduel, M., McGlinn, K., Pauwels, P., Priyatna, F., Wagner, A., Kukkonen, V., Steyskaland, S., Lehtonen, J., and Lefrançois, M. (2026). IFctoLBD (v2.46.0), github repository.
- Pauwels, P. (2020). IFctoRDF (v0.4), github repository.
- Pauwels, P. and Terkaj, W. (2016). Express to owl for construction industry: Towards a recommendable and usable ifcowl ontology. *Automation in Construction*, 63:100–133.
- Rasmussen, M. H., Lefrançois, M., Schneider, G. F., and Pauwels, P. (2021). BOT: The building topology ontology of the W3C linked building data group. *Semantic Web*, 12(1):143–161.